ML System Design Document Project Scope and Team Workflows

Retrieval Augmented Product Question Answering

This document defines the scope, team workflow, end-to-end ML architecture, release process, security controls, and operating plan for an evidence-based product question-answering prototype. The design is feasible in an AWS Academy Learner Lab with a $100 budget by using a limited dataset, CPU-first models, temporary SageMaker endpoints, manual labeling, and strict resource cleanup.

Team Information

Field	Value
Project Team Group	Group 8
Authors	Manoj Patra and Raminder Singh
Business and System Name	Retrieval Augmented Product Question Answering
Document Version	1.0
Publication Date	September 14, 2026
Document Status	Design proposed for implementation

Team Workflows

Project Links

Resource	Current Value
GitHub Project	https://github.com/ramindersinghusd/aai-540-g8
Asana Board	https://app.asana.com/1/952672460738672/project/1218498223094741/list/1218492250151155
Team Tracker	https://docs.google.com/document/d/1GXlsxEj9UYnFXqV_6zJ8iyszO7Hk56L1p2A-P1Wi-0A/edit?usp=sharing
AWS Region	Proposed primary region: us-east-1
Model Registry Group	product-qa-ranker, if Learner Lab permissions allow

Development Workflow

The team will keep the main branch deployable and use short-lived feature, fix, and experiment branches. Every pull request will link to a tracked task and state the purpose, tests, affected data or model artifacts, measured results, expected AWS cost, and rollback procedure. Manoj and Raminder Singh will review each other's work before merge.

1. Create an issue with an owner, acceptance criteria, and estimated AWS cost.
2. Implement the change on a short-lived branch and run local tests.
3. Open a pull request with evidence from data, model, or application tests.
4. Obtain review from the second team member and merge after CI passes.
5. Run the approved data, training, or deployment pipeline.
6. Record the dataset, code, container, model, and index versions in the team tracker.
7. Stop or delete billable compute immediately after the work session or demonstration.

Team Responsibilities

Activity	Responsible Role	Approver	Required Output
Project scope and acceptance criteria	Manoj Patra, Project Lead	Raminder Singh	Approved requirements and acceptance criteria
Data ingestion and validation	Raminder Singh, Data Engineering Lead	Manoj Patra	Versioned dataset, schema, and data-quality report
Data preprocessing and feature engineering	Raminder Singh, Data Engineering Lead	Manoj Patra	Reproducible processing pipeline and feature definitions
Retrieval and ranking model development	Manoj Patra, ML Engineering Lead	Raminder Singh	BM25 baseline, semantic retriever, and trained ranking model
Model evaluation and error analysis	Manoj Patra, ML Engineering Lead	Raminder Singh	Evaluation report, baseline comparison, and error analysis
AWS infrastructure and CI CD pipeline	Raminder Singh, MLOps Lead	Manoj Patra	Reproducible SageMaker pipeline and AWS resources
Flask API and web application	Manoj Patra, Application Engineering Lead	Raminder Singh	Tested API and deployed demonstration interface
Security and privacy review	Raminder Singh, Security and Data Review Owner	Manoj Patra	Completed security, privacy, and risk checklist
Integration and release testing	Raminder Singh, Quality Assurance Lead	Manoj Patra	Test report and completed release checklist
Model registration and deployment	Manoj Patra, MLOps Lead	Raminder Singh	Approved model version and deployment record
Monitoring dashboard and alarms	Manoj Patra, MLOps Lead	Raminder Singh	CloudWatch dashboard, baseline, and tested alarms

Activity	Responsible Role	Approver	Required Output
Documentation and presentation	Manoj Patra and Raminder Singh, Joint Owners	Manoj Patra and Raminder Singh	Final design, model card, runbook, and demonstration

Project Scope

Project Background

We will build a product question-answering system for an e-commerce setting. Given a customer question and product identifier, the system will retrieve relevant product attributes, descriptions, prior questions, community answers, or reviews and return a concise answer supported by that evidence. Each response will include the supporting source, confidence score, model version, and index version. When the evidence is insufficient or contradictory, the system will return a no-answer response.

The primary machine learning problem is supervised evidence relevance ranking within a retrieval-augmented architecture. The minimum viable product will return the highest-ranked existing answer or evidence passage. Constrained answer generation is an extension and will only run after retrieval, ranking, confidence filtering, and groundedness checks. The first release will support English questions and a limited product-category subset.

Technical Background

The system will use two retrieval stages. BM25 and sentence embeddings will produce a candidate set. A supervised learning-to-rank model will then combine semantic similarity, lexical overlap, source type, question type, and candidate-quality features. BM25 supplies a reproducible baseline. A frozen MiniLM sentence encoder and FAISS index provide semantic retrieval without the cost of transformer fine-tuning.

The primary ranker will be LightGBM LambdaRank. Logistic regression will provide an interpretable baseline, and a compact cross-encoder will remain a stretch experiment. Model selection will use held-out questions and relevance labels. The main metrics will be Precision at K, Recall at K, Mean Reciprocal Rank, and NDCG at K. Optional generated answers will be evaluated separately for correctness, usefulness, groundedness, and unsupported claims.

Goals

- Build a reproducible pipeline for ingesting, cleaning, splitting, featurizing, and versioning product question-answering data.
- Compare BM25 retrieval with semantic retrieval and supervised reranking.
- Train a model that ranks evidence for each product question and abstains when confidence is low.
- Deploy a real-time prototype and a repeatable offline evaluation workflow.

- Define monitoring baselines, release tests, cost controls, and a model approval process.

Non Goals

- Training a large language model from scratch.
- Building a complete shopping website or enterprise customer-support platform.
- Supporting every product category, language, or traffic pattern in the first release.
- Generating answers without retrieved evidence.
- Providing multi-region availability, enterprise autoscaling, or formal production service guarantees.

Functional Requirements

ID	Requirement
FR1	Accept an English product question and product identifier or context.
FR2	Retrieve and rerank candidate evidence for the requested product.
FR3	Return the best answer with evidence, source type, and confidence.
FR4	Return an explicit no-answer result below the approved threshold.
FR5	Support offline evaluation, batch index rebuilding, and reproducible runs.
FR6	Associate every response with model, preprocessing, and index versions.
FR7	Collect optional helpful, not helpful, and unsupported feedback.

Learner Lab Constraints

- The total AWS budget is $100, with an internal operating target of $60 and the remainder reserved for failed jobs and the final demonstration.
- The build will use a category subset and CPU-first models. GPU work is optional and capped at three to four hours.
- The SageMaker endpoint will exist only during integration testing and the final demonstration.
- Feature Store, Amplify, Pipelines, and Model Registry are conditional on the permissions assigned to the Learner Lab.
- S3 manifests and EC2-hosted inference provide fallbacks when a managed service is blocked.

Solution Overview

Public data will be copied into a private, versioned S3 raw area. SageMaker Processing will validate the schema, clean the text, remove duplicates, produce deterministic splits, and create candidate evidence groups. A CPU embedding job will create MiniLM vectors and a FAISS index. Training jobs will compare BM25, dense similarity, logistic regression, and LightGBM. A candidate will be registered only when it passes the defined quality, latency, security, and budget gates.

During normal development, Flask will load the approved model and FAISS index on a small EC2 instance. A temporary SageMaker endpoint will be created for final integration testing when Learner Lab permissions allow it. CloudWatch will receive operational and custom model-quality metrics. The team will use manual or daily monitoring jobs rather than continuous paid monitoring.

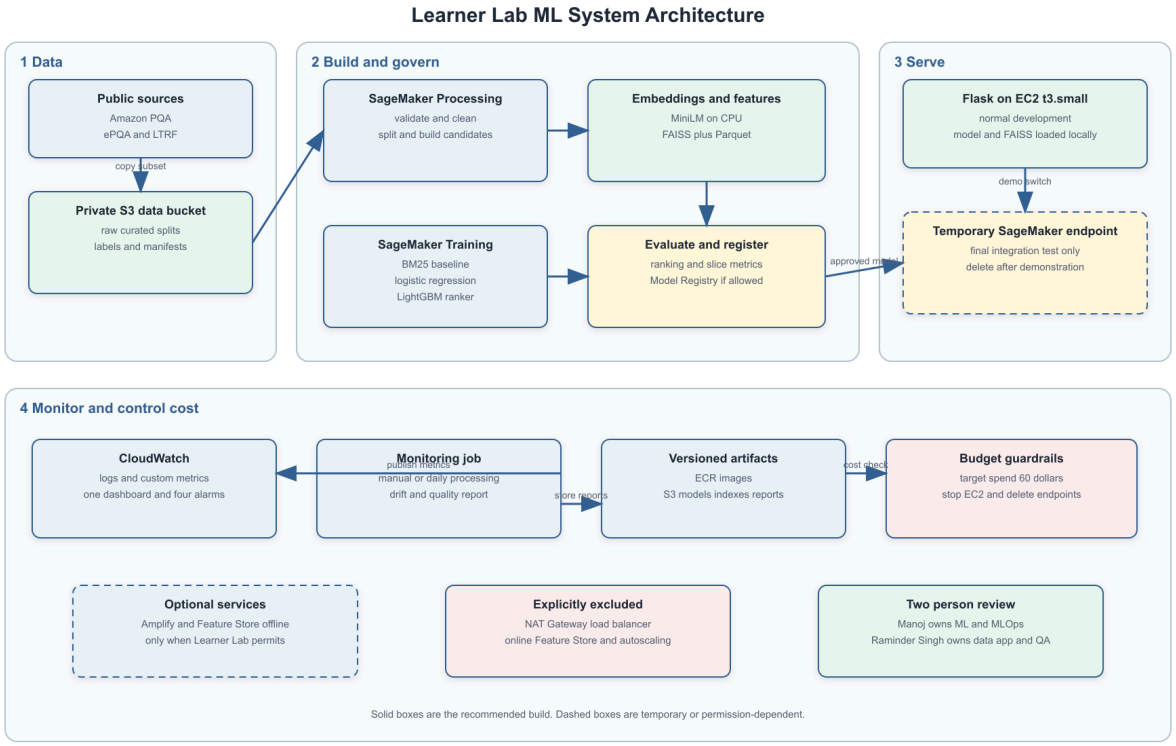


Figure 1 Learner Lab ML System Architecture

AWS Service Plan

RFC Service	MVP Decision	Learner Lab Implementation
S3 bucket	Use	One private bucket with raw, curated, artifact, and monitoring prefixes if bucket count is restricted.
SageMaker notebook	Use	ml.t3.medium or ml.m5.large, stopped when not in use.
SageMaker Feature Store	Conditional	Offline store only; versioned Parquet in S3 is the fallback.
Container registry	Use	ECR with only the latest two or three image versions retained.
CloudWatch	Use	Limited logs, one dashboard, and four alarms.
Autoscaling	Omit	Document as a production enhancement.
SageMaker endpoint	Temporary	Create for final tests and demonstration, then delete.
SageMaker Ground Truth	Conditional	Manoj and Raminder Singh label a small sample manually; no paid workforce.
EC2 instance	Use	t3.small for Flask and local inference; stop after each session.
Amplify	Conditional	Use if allowed; otherwise serve a basic interface from Flask.
Flask server	Use	REST API and demonstration interface.

Data Sources

Source	Content and Volume	Planned Use
Amazon PQA	Product questions, answers, ASINs, bullet points, descriptions, brands, and item names. The full archive is 2.1 GB compressed and 17.3 GB uncompressed.	Use one category and no more than approximately 25,000 to 50,000 usable records for the first experiment.
ePQA	Three-level relevance labels, evidence source, context, and manually written answers. The English release has 131,520 train and development pairs and 20,142 test pairs.	Primary supervised ranking and answer-evaluation dataset.
hetPQA	Evidence from six heterogeneous source types.	Secondary source-type analysis if time permits.
LTRF CQA	Product questions, answers, and answer-quality labels.	Optional answer-quality filtering experiment.
Team labels	A reviewed sample produced by Manoj and Raminder Singh.	Validate weak labels, abstention, and groundedness.

ePQA is the primary training source because it supplies explicit relevance labels. Amazon PQA supplies a larger retrieval corpus and product metadata. The category subset keeps embedding time, processing time, and storage within the Learner Lab budget.

Data Risks

- Public answers may be subjective, outdated, incorrect, or inconsistent.
- User-generated text may contain incidental personal information or unsafe language.
- Popular categories and products may be overrepresented.
- Random row-level splitting could leak near-duplicate products or questions.
- Weak negative sampling may incorrectly label useful evidence as irrelevant.
- Historical datasets must not be used as evidence for current price, inventory, warranty, or safety information.

Data Engineering

Storage Layout

S3 Prefix	Contents
raw/	Immutable source files, source metadata, and checksums
quarantine/	Malformed or policy-rejected records and reason codes
curated/	Normalized Parquet datasets
splits/	Immutable train, validation, and test manifests
features/	Scalar features and optional Feature Store exports
embeddings/	Question and evidence vectors
indexes/	FAISS index, metadata, and compatibility manifest
models/	Model artifacts and evaluation files
monitoring/	Sanitized inference metrics, baselines, and reports

Processing Steps

1. Copy the selected source files to the immutable raw prefix and record their checksums, license, size, and source date.
2. Validate the required columns, types, label values, and parse success rate.
3. Normalize Unicode, whitespace, HTML, product identifiers, categories, and source names.
4. Remove empty rows and exact duplicates, and group near-duplicates before splitting.
5. Detect English text and scan for email, telephone, address, and payment-card-like patterns.

6. Create candidate groups, weak negatives, hard negatives, and deterministic manifests.
7. Generate scalar features, MiniLM embeddings, and the versioned FAISS index.
8. Publish a data-quality report and stop the pipeline if a blocking rule fails.

Exploratory Analysis

- Rows, unique questions, products, categories, and candidates per question.
- Label balance, positive-to-negative ratio, and evidence-source distribution.
- Question, candidate, context, and answer lengths.
- Missing fields, duplicates, language, and potentially sensitive text.
- Question types such as compatibility, specifications, dimensions, quality, and yes or no.
- Similarity across splits and model performance by category, source, type, and answerability.

Training Data

The official ePQA train, development, and test files will be preserved for benchmark comparability. The ePQA test set will not be used for feature, model, or threshold selection. Amazon-PQA records will use a deterministic 70 percent training, 15 percent validation, and 15 percent test split grouped by ASIN. Near-duplicate questions and candidates will remain in the same split.

ePQA labels map to irrelevant, helpful but incomplete, and fully answering. Existing Amazon-PQA question-answer associations provide initial positive examples. Negative candidates will come from the same product or category, including difficult BM25 and embedding results. Manoj and Raminder Singh will each label 100 to 200 pairs and double-label at least 50 shared examples to measure agreement and resolve ambiguous cases.

Leakage Controls

- Fail when a question identifier or Amazon-PQA ASIN appears in multiple splits.
- Keep near-identical normalized pairs within one split.
- Exclude relevance labels and reference answers from ranker input features.
- Exclude future feedback and fields unavailable at online inference time.
- Keep test records out of feature, threshold, and hyperparameter selection.

Feature Engineering

Feature Group	Examples	Treatment
Text	Question, candidate, context, and product title	Normalize text; preserve negation, measurements, model numbers, and units.
Semantic	MiniLM embeddings and cosine similarity	L2 normalize and compute with the same model version in training and inference.

Feature Group	Examples	Treatment
Lexical	BM25 score, token overlap, phrase and number match	Use as baseline and ranker features.
Metadata	Category, source type, and question type	Encode as categorical features.
Length and quality	Token counts, missing text, duplicate and boilerplate flags	Use for filtering and ranking.
Identifiers	qid, pair ID, ASIN, ingestion version	Use as keys and split controls, not predictive features.
Sensitive data	Names, contact data, payment-like text	Redact or exclude unless required for a documented safety test.

Scalar features will be stored as Parquet in S3 and optionally in the Feature Store offline store. Dense vectors and candidate metadata will be packaged in the FAISS index because they are retrieval artifacts rather than ordinary tabular features.

Model Training and Evaluation

Model Stack

1. BM25 lexical retrieval baseline.
2. Frozen sentence-embedding retrieval using cosine similarity.
3. Logistic regression supervised baseline over retrieval and metadata features.
4. LightGBM LambdaRank as the primary ranking model.
5. Compact transformer cross-encoder as an optional experiment if time and budget remain.
6. Constrained evidence-only answer generation as a future extension.

Initial Parameters

Component	Initial Configuration
Embedding model	all-MiniLM-L6-v2 or equivalent compact encoder
Embedding dimension	384
Retrieved candidates	Top 50
Reranked candidates	Top 10
Returned evidence	Top 3
LightGBM objective	lambdarank
Learning rate	0.05

Component	Initial Configuration
Number of leaves	31
Maximum estimators	500
L2 regularization	1.0
Early stopping	50 rounds
Random seed	42

Evaluation and Release Gates

Area	Metric	Initial Gate
Retrieval	Recall at 10	At least 0.85 and 0.05 above BM25
Ranking	Mean Reciprocal Rank	At least 0.05 absolute improvement over BM25
Ranking	NDCG at 10	At least 0.03 absolute improvement over BM25
Top answer	Precision at 1	No regression from the best approved model
Confidence	Expected calibration error	No greater than 0.10
Abstention	No-answer precision	At least 0.90 on the reviewed set
Groundedness	Unsupported claim rate	No greater than 5 percent
Serving	Extractive p95 latency	No greater than 2.5 seconds
Serving	Error rate	Less than 1 percent in the load test

The thresholds are initial design targets and will be recalibrated after the baseline run. The evaluation report will include bootstrap confidence intervals and slices by category, evidence source, question type, text length, and answerability. A higher aggregate score will not override a serious regression in a high-risk slice.

Model Deployment

Deployment Pattern

The user-facing application needs interactive responses, but an always-running SageMaker endpoint is not affordable under the lab budget. During routine development, a t3.small EC2 instance will run Flask, the MiniLM encoder, the FAISS index, and the approved ranker. For final integration testing, the team may deploy the same inference container to a SageMaker real-time endpoint for 12 to 20 total hours and delete it immediately afterward.

Workload	Initial Compute	Cost Control
Exploration	SageMaker ml.t3.medium	Stop when unused; target 60 to 80 hours total.
Data processing	ml.m5.large or ml.m5.xlarge	Run on the selected category only.
BM25 and LightGBM	ml.m5.xlarge	CPU training with early stopping.
Optional transformer	ml.g4dn.xlarge	Maximum three to four hours.
Routine inference	EC2 t3.small	Stop after each work session.
Final endpoint test	SageMaker ml.m5.large or ml.m5.xlarge	Temporary deployment only.

Online Inference Flow

1. The client sends a product identifier, English question, and unique request identifier to Flask.
2. Flask validates the schema, payload size, and request rate.
3. The inference service normalizes and embeds the question.
4. FAISS retrieves the top 50 candidates, restricted by product or permitted category context.
5. The service calculates lexical, semantic, source, length, and quality features.
6. LightGBM reranks the top 10 candidates and produces a calibrated confidence score.
7. The system returns the best extractive answer and up to three evidence passages, or abstains below the threshold.
8. The API records sanitized metrics, model version, index version, latency, and optional feedback.

The model package will include a compatibility manifest binding the model to its preprocessing version, feature schema, embedding model, and FAISS checksum. Deployment will fail when those versions do not match. Rollback will restore the previous approved model and its associated index.

Model Monitoring

A scheduled or manually executed SageMaker Processing job will compare sanitized inference samples with the approved validation baseline. This custom approach is suitable for a Learner Lab because SageMaker Model Monitor access may be unavailable and AWS no longer opens Model Monitor to new customers. The job will publish custom quality and drift metrics to CloudWatch and store the detailed report in S3.

Area	Metrics	Initial Alert
Infrastructure	CPU, memory, disk, endpoint state	Resource use above 80 percent for 15 minutes

Area	Metrics	Initial Alert
API	Requests, p50 and p95 latency, 4xx and 5xx	5xx above 1 percent or p95 above 2.5 seconds
Input quality	Missing product, empty question, invalid schema	Invalid request ratio above 2 percent
Drift	Category, topic, length, and embedding distributions	Population Stability Index above 0.20
Retrieval	Top score, score margin, Recall at 10, MRR	Metric drop above 5 percentage points
Answerability	Confidence and no-answer rate	No-answer rate exceeds twice the baseline
Groundedness	Unsupported or contradictory answer rate	Above 5 percent in the reviewed sample
Feedback	Helpful rate and reported errors	Helpful rate drops by 10 percentage points
Cost	Notebook, EC2, processing, training, and endpoint hours	Projected total exceeds the $60 operating target

Model CI CD

SageMaker Pipelines will orchestrate processing, embedding, training, evaluation, and registration when the Learner Lab role permits the required actions. A scripted sequence using the same manifests and quality checks will provide a fallback. Automatic training may create a candidate, but promotion requires review by the second team member.

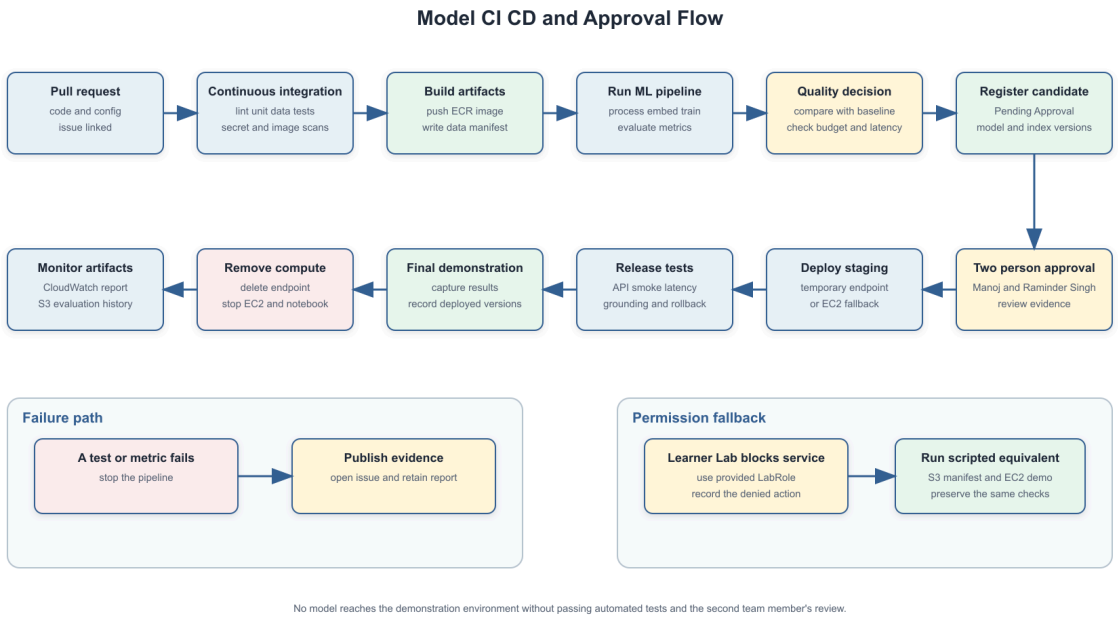


Figure 2 Model CI CD and Approval Flow

Release Tests

Checkpoint	Required Evidence
Pull request	Formatting, linting, unit tests, secret scan, and dependency scan
Ingestion	Schema, checksum, parsing, required columns, and license manifest
Preparation	Null, duplicate, language, label, and split-integrity tests
Features	Ranges, deterministic output, and training-inference parity
Retrieval	Known-query tests, product filtering, and index checksum
Model	Reproducibility, baseline comparison, slices, and calibration
Safety	Grounding, contradiction, PII, unsafe text, and prompt-injection tests
Container	Image scan, health route, non-root process, and version compatibility
Staging	API contract, smoke, latency, load, failure handling, and rollback
Release	Second-member approval, monitoring enabled, and cost checked

Security Privacy and Other Risks

Security Checklist

Question	Decision and Control
Will the system process PHI	No PHI is intentionally collected. Health-related text is untrusted and will not be used for medical advice.
Will the system process PII	Potentially. Public answers and live questions may contain incidental PII. Redact it before monitoring or long-term storage.
Will user behavior be tracked	Only minimum operational telemetry and explicit feedback. No cross-session user profile is planned.
Will credit card data be stored	No. Reject or redact payment-card-like text.
Why retain any user data	Sanitized samples and feedback support debugging, monitoring, and reviewed evaluation.
Which S3 resources are read	Public Amazon-PQA and LTRF sources plus the private project data prefix.
Which S3 resources are written	Private project data, artifacts, and monitoring prefixes only.
Are protected attributes features	No. They are excluded from planned predictive features.

Security Controls

- Use the provided LabRole and least-privilege policies available in the class environment.

- Block public access to project buckets and encrypt stored artifacts.
- Use TLS and keep credentials out of source code and notebooks.
- Validate payload size and type, escape displayed evidence, and rate-limit the public demonstration.
- Retain only sanitized logs for a limited period.
- Scan dependencies and ECR images and keep only required image versions.
- Treat retrieved passages as untrusted data and never as executable instructions.

Bias and Ethical Risks

The model does not require demographic attributes, but it can inherit bias from category coverage, product popularity, language, and subjective community answers. Performance will be reported by category, source, question type, product popularity, and answerability. A small counterfactual set will check whether irrelevant demographic terms change the ranking.

Incorrect compatibility, electrical, child-safety, or product-use answers can create financial or physical harm. The interface will display evidence, distinguish community opinions from manufacturer information, preserve measurements and negation, and abstain when evidence is missing or contradictory. Historical data will not support claims about current price, inventory, warranty, or regulatory status.

AWS Learner Lab Budget Plan

The core project is feasible within the $100 Learner Lab budget. The team will treat $60 as the operating limit and reserve the remaining $40 for failed jobs, permissions workarounds, and the final demonstration. Costs below are planning estimates for us-east-1 and should be checked against the lab's actual usage display.

Service	Planned Usage	Estimated Cost
S3	Up to 30 GB for two months	$1 to $2
ECR	Up to 5 GB of images	Less than $1
SageMaker notebook	60 to 80 hours on ml.t3.medium	$3 to $6
SageMaker Processing	15 to 20 CPU hours	$4 to $6
SageMaker Training	20 to 30 CPU hours	$5 to $10
Optional GPU	No more than four hours	$3 to $5
SageMaker endpoint	12 to 20 demonstration hours	$3 to $6
EC2 Flask server	80 to 120 hours on t3.small	$2 to $4
EBS	10 to 20 GB	$1 to $2
CloudWatch	Small logs, dashboard, and alarms	$2 to $5

Service	Planned Usage	Estimated Cost
Amplify	Small optional static application	$0 to $3
Requests and transfer	Small academic workload	$0 to $3
Contingency	Failed jobs and reruns	$15 to $20
Expected total	Project estimate	$39 to $72

Primary Cost Risk

The SageMaker endpoint is the largest budget risk. At an indicative $0.23 per hour, an ml.m5.xlarge endpoint running continuously for 30 days would cost about $165.60 before other services: $0.23 x 24 hours x 30 days. The endpoint must therefore be temporary. Stopping a notebook does not stop an independent endpoint.

Budget Guardrails

1. Use one AWS Region and check the Learner Lab balance after every compute session.
2. Stop notebook applications and the Flask EC2 instance when work ends.
3. Delete SageMaker endpoints and endpoint configurations after testing.
4. Limit the initial Amazon-PQA subset to approximately 25,000 to 50,000 records.
5. Retain only the latest two or three ECR images and apply S3 lifecycle rules to temporary outputs.
6. Do not create a NAT Gateway, Application Load Balancer, OpenSearch domain, RDS database, online Feature Store, or always-on GPU instance.
7. Tag billable resources with Project, Owner, and DeleteAfter values.
8. Do not start an optional transformer or generation experiment unless at least $40 remains.

Implementation Plan

Phase	Manoj	Raminder Singh	Exit Evidence
Foundation	Repository, LabRole checks, ECR, CI	Task board and application skeleton	Infrastructure and CI checks pass
Data	Review architecture and cost	Ingestion, schema, preprocessing, EDA	Quality and leakage tests pass
Baseline	BM25 and evaluation pipeline	Data and API test fixtures	Reproducible baseline report
Candidate model	Embeddings, FAISS, LightGBM	Feature validation and error review	Candidate exceeds baseline gates
Application	Model packaging and deployment	Flask interface and integration tests	End-to-end staging tests pass

Phase	Manoj	Raminder Singh	Exit Evidence
Monitoring	Metrics, dashboard, and alarms	Feedback and security tests	Test alarms and runbook verified
Final release	Model card and deployment record	Release test report and demo	Both members approve submission

Definition of Done

- Code is merged through the two-person workflow and all required tests pass.
- Configuration is reproducible and contains no embedded credentials.
- Dataset, container, model, feature, and index versions are recorded.
- Evaluation results, limitations, costs, and rollback behavior are documented.
- Monitoring and cleanup procedures are tested.
- The README, model card, team tracker, and demonstration materials are current.

Future Enhancements

1. Evaluate a compact cross-encoder when additional GPU budget is available.
2. Add constrained generation with sentence-level evidence checks.
3. Use active learning to prioritize uncertain or high-impact examples for review.
4. Add multilingual retrieval and generation using xPQA.
5. Support incremental index updates and additional product categories.
6. Move Flask to a managed autoscaling platform and use a managed vector service for larger corpora.
7. Add online experiments for confidence thresholds, answer presentation, and user feedback.

References

[1] RFC ML Project Proposal Retrieval Augmented Product Question Answering. Supplied project proposal.

[2] Amazon PQA Registry of Open Data

[3] Amazon PQA Data Documentation

[4] Learning to Rank and Filter Community Question Answering

[5] Amazon Science Contextual Product Question Answering

[6] ePQA and xPQA Dataset Paper

[7] AWS Academy Learner Lab

[8] Amazon SageMaker AI Pricing

[9] Amazon EC2 T3 Instances

[10] Amazon S3 Pricing

[11] Amazon ECR Pricing

[12] Amazon SageMaker Pipelines

[13] Amazon SageMaker Feature Store Concepts

[14] Amazon SageMaker Model Registry

[15] Amazon SageMaker Model Monitor Data Quality